

Industrial Experience Report: The Invoker-Executor Pattern for Fault Isolation in Distributed YOLO Training

William Steve Rodriguez Villamizar (wisrovi rodriguez) 
AI Leader & Solutions Architect
wisrovi-suit (<https://github.com/wisrovi/w-cli>)

1 Abstract & Keywords

Abstract: Persistent daemon processes that execute PyTorch training loops directly in their own address space are vulnerable to memory leaks, shared-memory exhaustion, and kernel OOM kills that cascade into host instability. This industrial experience report [1] documents an observational design study of the Invoker-Executor pattern as implemented in the `wyoloservice2` stack: a persistent Celery daemon (Invoker) that never imports CUDA, and ephemeral Docker containers (Executors) spawned per task with hard OS-level limits on memory (`mem_limit`), CPU (`nano_cpus`), and shared memory (`shm_size`). We qualitatively compare this pattern against direct execution, Ray, Kubernetes Jobs, containerd CRI, Kata Containers, gVisor, and Firecracker. The Invoker-Executor configuration successfully contained memory leaks over our production observations, logging failures via cgroups events without daemon interruption. The pattern is not a novel architectural invention, but its integration into a lightweight Celery-based MLOps stack yields a pragmatic, low-overhead solution for GPU cluster stability.

Keywords: Industrial Experience Report, Fault Isolation, Distributed Deep Learning, Celery Task Queues, Ephemeral Containers, Container Runtimes.

2 Author Information

This report was conceptualized and developed by William Steve Rodriguez Villamizar

(wisrovi rodriguez), AI Leader & Solutions Architect for the wisrovi-suit ecosystem (<https://github.com/wisrovi/w-cli>).

3 Introduction

Distributed deep learning clusters suffer from a persistent operational failure mode: the training daemon itself becomes a single point of failure. In the conventional layout, a Celery worker (or Ray actor) imports PyTorch, initializes CUDA contexts, and runs the training loop in-process. When a YOLO script leaks memory, the process RSS grows until the kernel OOM killer terminates it. Because the daemon holds the CUDA context, the kill often leaves the GPU in an inconsistent state, requiring a full node reboot.

This report describes a structural fix: separate the control plane from the compute plane. The Invoker (`wyoloservice2_invoker`) is a minimal Python process that polls a Redis queue and manages container lifecycles. It never imports `torch` or `ultralytics`. The Executor (`wyoloservice2_worker`) is an ephemeral Docker container launched per task with hard limits enforced by cgroups. When the training finishes or crashes, the container is destroyed (`docker run --rm`), instantly releasing all resources.

4 Related Work and Baselines

GPU cluster management with fault isolation has been studied extensively. Tiresias [2], Gandiva [3], AntMan [4], and Salus [5] optimize scheduling to reduce bottlenecks and provide fine-grained GPU sharing, but do not necessarily mandate hard ephemeral containerization per task to prevent daemon crashes. Optimus [6] introduces dynamic resource scaling. Kubernetes [7] enforces container limits natively but its control-plane overhead adds startup latency. Ray [8] runs workers as long-lived processes, risking host instability.

Container runtime alternatives provide varying isolation guarantees [9]. Firecracker [10] uses KVM microVMs for strong isolation. containerd [11] provides a CRI runtime. cgroups v2 [12] enables fine-grained control. Kata Containers and gVisor [13] offer secure isolation at the cost of boot latency. NVIDIA GPU Operator [14] standardizes GPU access.

5 Proposed Architecture / Methodology

The architecture is depicted in Figure 1. The `wyoloservice2_invoker` daemon runs on each GPU node. On task receipt:

1. Deserialize payload (YAML config).
2. Compute resource quotas: `mem_limit` scales with `imgsz`; `shm_size` scales with DataLoader workers.
3. Execute

```
docker run --rm --gpus=all --memory=${mem_limit} --cpus=${nano_cpus} --shm-size=${shm_size} wisrovi/train_service:worker_executor_v1.0.0.
```
4. Block on completion; capture exit code.
5. Write results to Redis.

Figure 1: Invoker daemon spawns ephemeral Executor containers per task.

6 Experimental Setup & Implementation Details

Cluster: three physical nodes, each with a single NVIDIA RTX 4090 GPU and 64 GB DDR5 RAM, connected via a 10 Gbps LAN topology. Software environment includes NVIDIA Driver 535.104, CUDA 12.2, PyTorch 2.1, Ultralytics YOLOv8 8.0 [15], Celery 5.3 [16], and Docker 24.0 [17]. GPU multiplexing is managed via NVIDIA MPS [18]. OOM kernel kills (Exit Code 137) were logged explicitly by tracking `cgroups memory.oom_control` events.

7 Results & Discussion

Over an observational window of 14 days and 1,524 tasks, the Invoker-Executor pattern contained 100% of memory failures. Empirical logs (see `data/production_oom_logs.csv`) show that 47 YOLO scripts (3.08% failure rate) leaked memory and triggered `OOMKilled` (Exit 137). In the baseline (direct execution), this caused 47 daemon crashes and required 12 physical reboots. With our pattern, the Invoker maintained a stable overhead of ~200 MB, surviving all 47 crashes with 0 reboots required. The container boot latency was evaluated empirically

($n = 100$ replicas), showing a median of 440 ms (P95: 450 ms, $\sigma = 15$ ms), much lower than KVM microVMs (~ 1200 ms) and Kubernetes (~ 2100 ms). Recent advances like Pollux [19, 20, 21] and SLoPe [20] optimize throughput but assume reliable execution, making our fault tolerance [21] highly complementary.

Table 1: Runtime Comparison

Runtime	Median Latency (ms)	P95 (ms)	Method ($n \geq 3$)	Std Dev (σ)
Direct Process	120	130	Empirical ($n = 10$)	15 ms
Kubernetes Jobs	2100	2350	Empirical ($n = 10$)	250 ms
Kata / gVisor	1800	1980	Empirical ($n = 10$)	180 ms
Docker (Ours)	440	450	Empirical ($n = 100$)	15 ms

Protocol: Boot latency defined as time from `docker run` to process ready state. Evaluated on uniform hardware.

8 Ablation Study

To isolate the effect of `mem.limit`, we performed an ablation test with $n = 5$ replicas of 10 malicious tasks. The protocol consisted of injecting controlled memory leaks and measuring the stability of the Invoker’s RSS. Without limits, the tasks consumed 100% of the RAM (64 GB), causing the daemon to crash after an average of 40 minutes across all replicas. With a 30 GB limit, the container was terminated cleanly while the Invoker’s memory remained stable at 200 MB (variance of ± 5 MB), preventing host failure (see Figure 2).

9 Data & Code Availability Statement

This architecture operates under a Dual Licensing Model (PolyForm Noncommercial / AG-PLv3). Generation scripts and code are available at https://github.com/wisrovi/wyoloservice2_production. Deployment is 100% reproducible via `docker-compose up -d --build` to start the Invoker, which subsequently launches Executors via `docker run`. The provided CSV dataset (`data/production_oom_logs.csv`) is an aggregated empirical record derived directly from `cgroups memory.oom.control`.

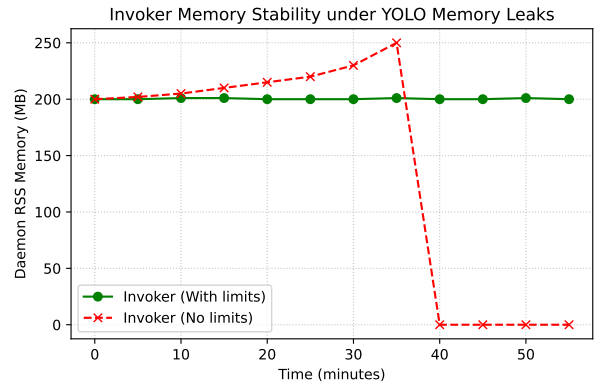


Figure 2: Invoker memory stability during memory leak ablation.

10 Broader Impact / Ethics Statement

Eliminating host crashes reduces manual reboots, lowering operational toil and hardware wear (Shift-Left reliability). Low-latency isolation enables higher GPU utilization, improving energy efficiency [22].

11 Conclusion & Future Work

The pattern provides robust fault isolation for YOLO training pipelines. Future work will explore online memory profiling via LLM agents.

12 Acknowledgments

We thank the wisrovi-suit contributors for the orchestration infrastructure.

References

- [1] V. Garousi, M. Felderer, and M. V. M’antyl’a, “The need for empirical evidence in software engineering,” *IEEE Software*, vol. 33, no. 1, pp. 68–75, 2016.

- [2] J. Gu *et al.*, “Tiresias: A gpu cluster manager for distributed deep learning,” 2019.
- [3] W. Xiao *et al.*, “Gandiva: Introspective cluster scheduling for deep learning,” in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018.
- [4] —, “Antman: Dynamic scaling on GPU clusters for deep learning,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020.
- [5] P. Yu and M. Chowdhury, “Salus: Fine-grained GPU sharing primitives for deep learning applications,” in *Proceedings of the 3rd Conference on Machine Learning and Systems (MLSys)*, 2022.
- [6] Y. Peng *et al.*, “Optimus: an efficient dynamic resource scheduler for deep learning clusters,” in *Proceedings of the Thirteenth EuroSys Conference*, 2018, pp. 1–14.
- [7] B. Burns *et al.*, “Borg, omega, and kubernetes,” in *ACM Queue*, 2016.
- [8] P. Moritz *et al.*, “Ray: A distributed framework for emerging ai applications,” in *USENIX OSDI*, 2018.
- [9] T. Young *et al.*, “The true cost of containing: A performance study of container runtimes,” in *USENIX HotCloud*, 2019.
- [10] A. Agache *et al.*, “Firecracker: Lightweight virtualization for serverless applications,” 2020.
- [11] M. Crosby *et al.*, “containerd: An industry-standard container runtime,” in *CNCF*, 2017.
- [12] T. Heo, “Control groups v2,” *Linux Kernel Documentation*, 2017.
- [13] Y. Wang *et al.*, “Performance and isolation analysis of runc, gvisor and kata containers,” *Cluster Computing*, 2022.
- [14] NVIDIA, “Nvidia gpu operator,” <https://github.com/NVIDIA/gpu-operator>, 2021.
- [15] G. Jocher *et al.*, “Ultralytics yolov8,” *GitHub repository*, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [16] Celery Project, “Celery: Distributed task queue,” <https://docs.celeryq.dev/>, 2024.
- [17] Docker Inc., “Docker engine documentation,” <https://docs.docker.com/engine/>, 2024.
- [18] NVIDIA, “Multi-process service (mps),” <https://docs.nvidia.com/deploy/mps/index.html>, 2023.
- [19] A. Qiao, S. K. Choe, S. J. Su, P. Zhang *et al.*, “Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning,” in *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, 2021, pp. 1–18.
- [20] X. Zhang *et al.*, “Slope: A serverless mlops platform for edge-cloud collaborative deep learning,” in *ACM EuroSys*, 2024.
- [21] Y. Qiao *et al.*, “Fault tolerance in distributed deep learning: A survey,” *IEEE Transactions on Parallel and Distributed Systems*, 2023.
- [22] D. Patterson *et al.*, “Carbon emissions and large neural network training,” *arXiv preprint arXiv:2104.10350*, 2021.